

Smart Learn LMS

Complete Project Guide

Combined guide containing the complete project documentation, root README, frontend README, and backend README.

Purpose: Team reference, competition presentation preparation, project understanding, setup, API reference, and technical documentation.

Project Type: Full-Stack MERN Learning Management System

Roles: Student • Teacher • Administrator • Parent QR Report Access

This guide intentionally retains the topics from all four supplied Markdown documents so that no documented topic is omitted.

□ Guide Contents

- 1. Complete Project Documentation — overall system, modules, architecture, workflows, database, APIs, deployment, testing, security, and presentation flow
- 2. Root README — repository overview, features, setup, health check, environment variables, API overview, project structure, roadmap, and admin credential management
- 3. Frontend README — React UI, Parent QR, components, state, routing, Axios, styling, responsive design, build/deployment, troubleshooting, performance, security, analytics, and contributing
- 4. Backend README — Node/Express API, architecture, security, threat mitigation, all backend modules, API documentation, database schema, authentication, file uploads, email, testing, deployment, maintenance, and Parent QR

Note: Some subjects appear in more than one source (for example authentication, deployment, Parent QR, and API documentation). They are retained where documented so the guide does not lose source-specific details.

Part I — Complete Project Documentation

Source: PROJECT_DOCUMENTATION_v2(1).md

Learning Management System (LMS) — Complete Project Documentation

Document Version: 2.0

Status: Competition / Final Presentation Version

Project Type: Full-Stack MERN Learning Management System

1. Executive Summary

The Learning Management System (LMS) is a full-stack web application designed to digitally manage academic activities in one centralized platform.

The system supports three primary roles:

- Student
- Teacher
- Administrator

The platform provides role-based authentication, course management, assessments, timetable management, learning resources, communication, personal study tools, an AI-powered Study Bot, and a Parent QR / Student Report feature.

The application is designed with a responsive interface so that important functionality can be accessed across desktop and mobile screen sizes.

2. Project Overview

2.1 Purpose

The main purpose of the LMS is to provide a centralized digital platform for managing teaching, learning, assessments, communication, and academic information.

Key goals include:

- Digital transformation of traditional educational management
- Centralized course and academic information
- Role-based access for students, teachers, and administrators
- Online assessments and result tracking
- Easy sharing of learning resources
- Communication between users
- AI-assisted study support
- Convenient parent access to student academic information through QR

2.2 Key Features

- Multi-Role Authentication: Separate student, teacher, and admin experiences
- Course Management: Course creation, assignment, viewing, and resource management
- Assessment System: MCQ and theory-based tests
- Result Management: Submission and performance tracking
- Timetable Management: Academic schedule organization
- Resource Management: Course learning materials and links
- Communication: Messaging, announcements, and course comments
- AI Study Bot: AI-assisted study support
- Notes & Reminders: Personal study organization
- Resource Library: Organized access to learning materials
- Parent QR: QR-based access to a student's academic report
- Responsive UI: Mobile and desktop-friendly interface

3. User Roles

3.1 Student

Students can:

- Register and log in
- View their dashboard
- View enrolled courses
- Access course resources
- View timetable
- Take available tests
- View test results
- Communicate through available messaging features
- Add personal notes
- Create reminders
- Use the AI Study Bot
- Access their resource library
- Display a Parent QR for sharing their academic report

3.2 Teacher

Teachers can:

- Log in using their account
- View their teaching dashboard
- View assigned courses
- Manage course resources

- Add course comments
- Create assessments
- Publish tests
- Grade theory submissions
- View student performance/results
- Send messages and announcements
- Manage their teaching timetable

3.3 Administrator

Administrators can manage major academic and user-level operations, including:

- Teacher account creation
- Student management
- User roles
- Course creation and assignment
- Student/teacher course assignment
- Timetable administration
- System-level result and user information

4. System Architecture

4.1 Technology Stack

Frontend

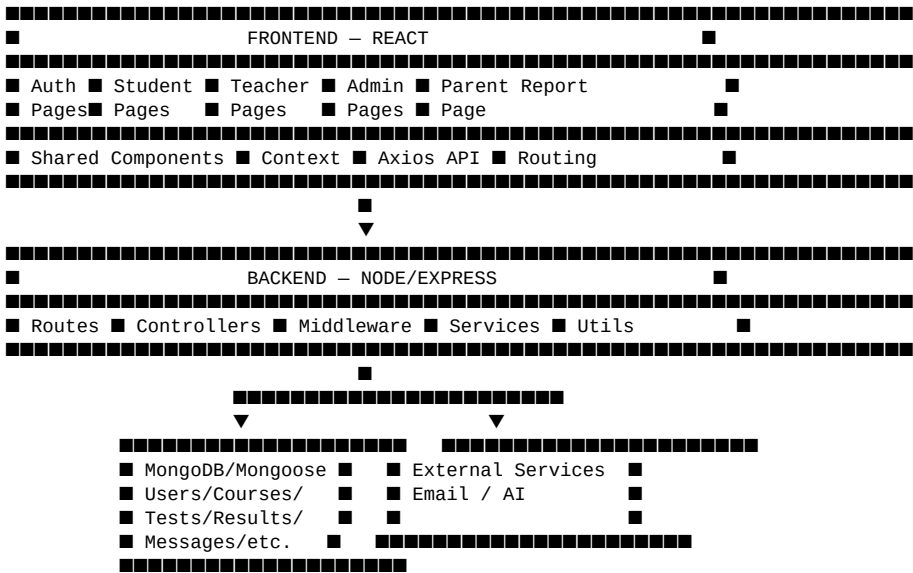
- React: User interface
- React Router DOM: Client-side routing
- Tailwind CSS: Responsive styling
- Axios: API communication
- React Icons: UI icons
- Vite: Frontend development and build tool
- qrcode.react: QR code generation

Backend

- Node.js: JavaScript runtime
- Express.js: Backend/API framework
- MongoDB: Database
- Mongoose: MongoDB ODM
- JWT: Authentication
- bcryptjs: Password hashing

- Multer: File upload handling
- Nodemailer: Email functionality
- Groq / AI model API: AI Study Bot integration

4.2 High-Level Architecture



5. Detailed Features

5.1 Authentication System

Registration

- Students can register through the registration flow.
- Teacher accounts are created through administration.
- Authentication uses user roles to provide the appropriate application experience.

Login

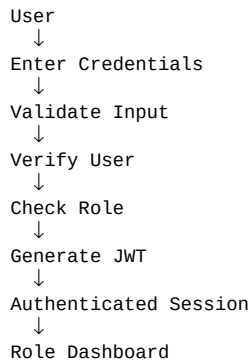
The application supports role-based login and authentication using credentials associated with the user account.

Password Management

The project includes:

- Forgot password flow
- Email OTP verification
- Password reset
- Change password functionality
- Show/hide password UI

Authentication Flow



6. Student Module

6.1 Student Dashboard

The dashboard provides students with a centralized view of their academic activity.

It includes:

- Today's classes
- Recent activities
- Course overview
- Quick actions
- AI Study Bot access
- Parent QR access

6.2 Course Management

Students can:

- View enrolled courses
- Read course information
- Access course resources
- View course-related content
- Participate in course comments where available

6.3 Timetable

The timetable module provides students with their academic schedule.

Features include:

- Personal timetable
- Day-wise organization

- Class information
- Responsive timetable UI

6.4 Assessment System

Students can:

- View available tests
- Open published assessments
- Answer MCQ and theory questions
- Submit assessments
- View available results

The assessment system supports automatic evaluation for applicable MCQ questions and teacher evaluation for theory-based responses.

7. Parent QR & Student Report

7.1 Overview

The Parent QR feature allows a student to share a QR code that can be scanned by a parent to open the student's academic report.

The QR system was added as an additional feature to make student information easier to access without requiring a separate parent login.

7.2 How It Works

```
graph TD
    A[Student Dashboard] --> B[Parent QR is displayed]
    B --> C[Parent scans QR]
    C --> D[Student Report URL opens]
    D --> E[Backend receives Student ID]
    E --> F[Student information is fetched]
    F --> G[Parent Report is displayed]
```

7.3 QR Code

The QR code is generated on the student dashboard using `qrcode.react`.

The QR contains a URL pointing to the student's report page.

The QR does not need to store the complete academic report data itself. Instead, it contains the URL with the student's identifier.

Example flow:

```
/parent-report/<studentId>
```


7.4 Parent Report

The standalone Parent Report page displays:

Student Details

- Student name
- Registration number
- Email

Enrolled Courses

- Course title
- Course description
- Teacher name

Test Results

- Test title
- Course
- Marks
- Percentage
- Grading status

The Parent Report page is publicly accessible through the QR flow and is designed as a standalone page without the normal student/teacher/admin dashboard navigation.

7.5 Backend Endpoint

```
GET /api/student/parent-report/:studentId
```

The backend uses the supplied student ID to retrieve the student and relevant academic information.

8. Teacher Module

8.1 Teacher Dashboard

Teachers can view:

- Teaching courses
- Scheduled classes
- Enrolled students
- Academic activity
- Performance-related information

8.2 Course Management

Teachers can:

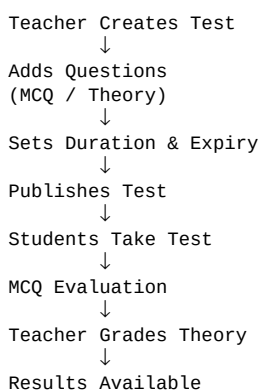
- View assigned courses
- Upload course resources
- Share files/links
- Manage course comments
- Monitor student progress where supported

8.3 Assessment Tools

Teachers can:

- Create tests
- Add MCQ questions
- Add theory questions
- Set duration
- Set expiry
- Publish tests
- Grade theory submissions
- Review results

Test Management Flow



8.4 Communication

Teacher communication features include:

- Messaging
- Announcements
- Message editing
- Course comments
- Student communication

9. Admin Module

9.1 User Management

Administrators can manage:

- Teacher creation
- Student records
- User roles
- User-related academic information

9.2 Course Administration

Admin operations include:

- Course creation
- Teacher assignment
- Student assignment
- Course editing
- Course-level management

9.3 Timetable Administration

Admin can manage:

- Class schedules
- Time slots
- Student/teacher timetable assignments
- Timetable-related operations

10. AI Study Bot

The LMS includes an AI-powered Study Bot to assist students with study-related activities.

The Study Bot provides an interactive AI-based interface inside the student experience.

It can be used as a study assistant for:

- Asking study-related questions
- Getting explanations
- Supporting learning
- Assisting with study notes and related tasks

The backend connects the application to an external AI model service.

11. Study Tools

11.1 Personal Notes

Students can maintain personal notes for organizing study information.

11.2 Reminders

The reminder feature helps students keep track of tasks and study deadlines.

11.3 Resource Library

The resource library provides an organized area for accessing learning resources.

12. Communication System

The application provides communication features for academic interaction.

Supported functionality includes:

- Messaging
- Teacher announcements
- Notifications where implemented
- Course comments
- Message management

13. Database Design

The backend uses MongoDB with Mongoose models.

Main entities include:

User
Course
Timetable
Test
Submission
Message
Note
Reminder

13.1 User

Stores user information such as:

```
{
  name,
  email,
  password,
  registrationNumber,
  role,
  emailVerified,
```

```
otp,  
otpExpires,  
createdAt,  
updatedAt  
}
```

Roles include:

```
student  
teacher  
admin
```

13.2 Course

A course contains information such as:

```
{  
  title,  
  description,  
  teacher,  
  students,  
  resources,  
  timetable,  
  comments,  
  createdAt,  
  updatedAt  
}
```

13.3 Test

A test contains:

```
{  
  title,  
  description,  
  course,  
  questions,  
  duration,  
  published,  
  expiryDate,  
  createdBy,  
  createdAt,  
  updatedAt  
}
```

Questions can support:

- MCQ
- Theory

13.4 Submission

Submissions connect students with tests and store assessment performance, including marks, maximum marks, percentage, grading state, and submission information.

14. API Overview

Authentication

```
POST /api/auth/login  
POST /api/auth/register
```

Student

```
GET /api/student/dashboard
GET /api/student/profile
GET /api/student/courses
GET /api/student/timetable
GET /api/student/parent-report/:studentId
```

Teacher

```
GET /api/teacher/dashboard
GET /api/teacher/courses
POST /api/teacher/upload-resource
POST /api/teacher/add-comment
GET /api/teacher/messenger
POST /api/teacher/messenger
```

Admin

```
POST /api/admin/create-teacher
GET /api/admin/teachers
GET /api/admin/students
POST /api/admin/assign-course
POST /api/admin/assign-timetable
```

The complete API surface is larger than this overview. These endpoints represent the main documented operations.

15. Parent QR API Flow

```
GET /api/student/parent-report/:studentId
↓
Read studentId
↓
Find Student
↓
Find Enrolled Courses
↓
Find Submissions
↓
Prepare Report Data
↓
Return JSON Response
↓
Parent Report Page
```

16. Frontend File Structure

```
lms-client/
├── src/
│   ├── components/
│   │   ├── shared/
│   │   │   ├── Navbar.jsx
│   │   │   ├── Sidebar.jsx
│   │   │   ├── Loader.jsx
│   │   │   └── ErrorBoundary.jsx
│   │   └── pages/
│   │       ├── auth/
│   │       ├── student/
│   │       ├── teacher/
│   │       ├── admin/
│   │       ├── parent/
│   │       └── ParentReport.jsx
│   ├── routes/
│   ├── context/
│   └── api/
```

```
■ ■■■ App.jsx
■ ■■■ main.jsx
■
■■■ package.json
■■■ vite.config.js
```

17. Backend File Structure

```
lms-server/
■■■ controllers/
■ ■■■ adminController.js
■ ■■■ studentController.js
■ ■■■ teacherController.js
■ ■■■ testController.js
■
■■■ models/
■ ■■■ User.js
■ ■■■ Course.js
■ ■■■ Timetable.js
■ ■■■ Test.js
■ ■■■ Submission.js
■ ■■■ Message.js
■ ■■■ Note.js
■ ■■■ Reminder.js
■
■■■ routes/
■ ■■■ auth.js
■ ■■■ student.js
■ ■■■ teacher.js
■ ■■■ admin.js
■
■■■ middleware/
■■■ utils/
■■■ services/
■■■ uploads/
■■■ App.js
■■■ setup-admin.js
■■■ package.json
```

18. Responsive Design

The frontend has been adapted for different screen sizes.

Responsive improvements include:

- Mobile-friendly navigation
- Responsive sidebar behavior
- Responsive dashboard cards
- Responsive forms
- Responsive tables
- Mobile-friendly course layouts
- Responsive teacher/admin pages
- Responsive Parent Report page
- Responsive QR display

The interface uses Tailwind CSS responsive utilities to adapt layouts across screen sizes.

19. Installation & Local Development

Prerequisites

- Node.js
- npm
- MongoDB / MongoDB Atlas
- Git

Backend Setup

```
cd lms-server  
npm install
```

Create a `.env` file and configure the required backend environment variables.

Then start the server:

```
npm run dev
```

Frontend Setup

```
cd lms-client  
npm install
```

Configure the frontend API URL in `.env`:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

Then run:

```
npm run dev
```

20. Deployment

The project can be deployed using separate frontend and backend hosting.

Current project deployment:

```
Frontend:  
Vercel
```

```
Backend:  
Render
```

```
Database:  
MongoDB Atlas
```

The frontend uses an environment variable for the deployed API:

```
VITE_API_BASE_URL=<backend-api-url>/api
```

21. Environment Variables

Typical backend configuration includes:

```
PORT=5000  
MONGO_URI=<mongodb-connection-string>  
JWT_SECRET=<jwt-secret>
```

```
EMAIL_HOST=<smtp-host>  
EMAIL_PORT=<smtp-port>  
EMAIL_USER=<email>  
EMAIL_PASS=<email-app-password>
```

```
GROQ_API_KEY=<ai-api-key>
```

Frontend:

```
VITE_API_BASE_URL=<backend-api-url>/api
```

Actual secret values should never be committed to GitHub.

22. Testing & Quality Assurance

Testing should cover:

Functional Testing

- Registration
- Login
- Password reset
- Course operations
- Test creation
- Test submission
- Result display
- Messaging
- Resource access
- AI Study Bot
- Parent QR scanning and report loading

Role Testing

Verify that:

- Student sees student features
- Teacher sees teacher features
- Admin sees admin features
- Unauthorized users cannot access protected role features

Responsive Testing

Test the interface on:

- Desktop

- Tablet
- Mobile

QR Testing

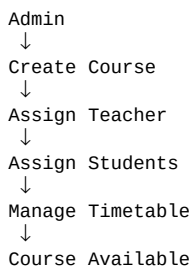
The Parent QR should be tested by:

1. Opening the student dashboard
2. Displaying the QR
3. Scanning it with a phone
4. Opening the generated report URL
5. Confirming student details load
6. Confirming courses/results display when data exists

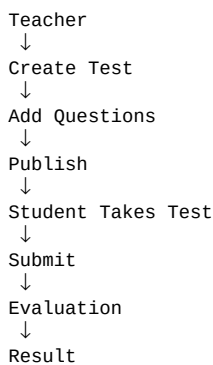
When testing a locally running frontend, a QR containing localhost generally cannot be opened from another phone because localhost refers to the phone itself. The deployed frontend URL should be used for real phone testing.

23. Important Project Workflows

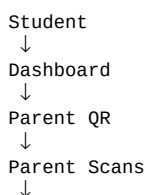
23.1 Course Assignment



23.2 Assessment



23.3 Parent Report



24. Security & Access Control

The project uses role-based authentication and protected application routes.

Implemented security-related concepts include:

- JWT-based authentication
- Password hashing with bcryptjs
- Role-based authorization
- Protected routes
- Environment variables for secrets
- Mongoose-based database interaction
- Input/error handling in API operations

Security configuration should be reviewed before any production-scale deployment.

25. Future Enhancements

Possible future improvements include:

- Dedicated parent login/account system
- Advanced parent dashboard
- Real-time chat
- Video conferencing
- Mobile application
- Advanced analytics
- AI-powered personalized recommendations
- Push notifications
- Improved reporting and attendance management
- Redis caching
- Docker containerization
- CI/CD pipeline
- Swagger/OpenAPI documentation

26. Presentation / Demo Flow

For a college competition demonstration, the recommended flow is:

1. Problem Statement
↓

2. Project Introduction
- ↓
3. User Roles
- ↓
4. Student Dashboard
- ↓
5. Course & Timetable
- ↓
6. Assessment & Results
- ↓
7. AI Study Bot
- ↓
8. Teacher Module
- ↓
9. Admin Module
- ↓
10. Parent QR Demo
- ↓
11. Technology Stack
- ↓
12. Future Scope

Highlight During Demo

The Parent QR can be presented as a distinctive feature:

“A student can display a Parent QR from the dashboard. A parent can scan it using a phone and directly access a standalone student report containing student information, enrolled courses, and assessment performance.”

27. Key Achievements

- Role-based LMS architecture
- Student, Teacher, and Admin modules
- Course management
- Online assessment system
- Result tracking
- Timetable management
- Learning resource management
- Communication features
- AI Study Bot
- Personal notes and reminders
- Responsive interface
- Parent QR and Student Report feature
- Frontend/backend separation
- MongoDB-based data management
- Deployable web application architecture

28. Conclusion

The Learning Management System provides a centralized digital platform for managing important academic activities.

The project combines course management, assessments, timetables, resources, communication, study tools, AI assistance, and parent-oriented academic access in one application.

The addition of the Parent QR and Student Report feature extends the platform beyond student/teacher/admin workflows and provides a simple way for parents to access relevant academic information.

The modular MERN architecture also provides a foundation for future improvements such as dedicated parent accounts, mobile applications, advanced analytics, real-time communication, and additional AI capabilities.

Document Information

Document Version: 2.0

Project: Learning Management System (LMS)

Status: Competition / Presentation Version

Prepared For: LMS Development Team

Part II — Root README

Source: README_root_updated(1).md

📦 Smart Learn — Learning Management System

A full-stack Learning Management System built with React + Vite and Node.js + Express + MongoDB, supporting three user roles: Students, Teachers, and Admins. It includes an AI-powered Study Bot, assessments, course management, communication tools, responsive dashboards, and a Parent QR-based Student Report.

Image: React

Image: Node.js

Image: MongoDB

Image: License

📸 Screenshots

🔗 Click to view screenshots

Login

Image: Login

Student Dashboard

Image: Student Dashboard

AI Study Bot

Image: Study Bot

Parent QR / Student Report

Add the final Parent QR / Parent Report screenshot here if you want it displayed in the repository README.

Student Tests

Image: Student Test

Results

Image: Results

Teacher Dashboard

Image: Teacher Dashboard

Teacher Test Management

Image: Teacher Test

Admin Dashboard

Image: Admin Dashboard

▢ Features

▢ Multi-Role Authentication

- JWT-based login for Students, Teachers, and Admins
- OTP email verification for password reset
- Role-based route protection on both frontend and backend

▢▢ Student

- Dashboard with enrolled courses, resources and today's classes
- Take MCQ and theory tests with timer
- View results and performance summary
- AI Study Bot powered by Groq
- Notes and reminders
- Timetable and messages from teachers
- Parent Access QR to open a standalone student academic report

▢▢ Teacher

- Dashboard with assigned subjects and class schedule
- Create, publish and manage tests (MCQ + theory)
- Grade theory submissions and give feedback
- Upload course resources (PDF, image, video)
- Messenger to communicate with students
- View student performance summaries

▢▢▢ Parent Access

- No parent account is required for the QR report flow
- Student generates a Parent Access QR from the Student Dashboard
- QR opens `/parent-report/:studentId`
- Standalone report displays student details, enrolled courses, teacher information, and assessment results

▢▢ Admin

- Create teacher accounts
- Create courses and assign teachers
- Enroll students into courses
- Assign timetables
- View all results across the platform
- Monitor system stats (users, courses, timetables)

▢ Current Project Status

The project is currently in the final testing / competition-presentation phase.

Live Deployment

- Frontend: <https://smart-lms-three.vercel.app/>
- Backend API: <https://smart-lms-kbqd.onrender.com/api>
- Database: MongoDB Atlas

Main completed areas

- Student, Teacher, and Admin modules
- Responsive UI across the main application pages
- AI Study Bot
- Tests and results workflow
- Teacher resources and communication
- Parent QR → standalone Student Report flow

📦 Tech Stack

Frontend

- React 18 + Vite
- React Router DOM
- Axios
- Tailwind CSS
- Framer Motion

Backend

- Node.js + Express 5
- MongoDB + Mongoose
- JWT Authentication
- bcryptjs
- Multer (file uploads)
- Nodemailer (OTP emails)
- Groq SDK (AI Study Bot)

Security

- MongoDB injection sanitization
- Secure HTTP headers
- JWT-based authentication and role-based access control
- Environment variables for secrets and API configuration
- Backend-side access checks for protected operations
- Environment-gated debug endpoints where configured

⚡ Getting Started

1. Clone the repo

```
git clone https://github.com/bs-bhaskar/smart-lms.git
cd smart-learn-lms
```

2. Backend setup

```
cd lms-server
npm install
cp .env.example .env
# Fill in your MONGO_URI, JWT_SECRET, and GROQ_API_KEY in .env
npm run setup-admin
npm run dev
```

Backend runs on: <http://localhost:5000>

3. Frontend setup

```
cd lms-client
npm install
cp .env.example .env
# Set VITE_API_BASE_URL=http://localhost:5000/api
# Production: VITE_API_BASE_URL=https://smart-lms-kbqd.onrender.com/api
npm run dev
```

Frontend runs on: <http://localhost:5173>

□ Health Check

The backend exposes a health endpoint to check server and database status:

```
GET http://localhost:5000/health
```

Response when healthy:

```
{
  "status": "ok",
  "database": "connected"
}
```

Response when database is down:

```
{
  "status": "degraded",
  "database": "disconnected"
}
```

□ Environment Variables

`lms-server/.env`

```
PORT=5000
NODE_ENV=development
MONGO_URI=your_mongodb_atlas_uri
JWT_SECRET=your_jwt_secret
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password
GROQ_API_KEY=your_groq_api_key
ENABLE_DEBUG_ENDPOINTS=false
ADMIN_NAME=Your Name
ADMIN_EMAIL=your_email@gmail.com
ADMIN_PASSWORD=your_password
```

`lms-client/.env`

```
VITE_API_BASE_URL=http://localhost:5000/api
```

API Overview

Auth

Method	Endpoint	Access
POST	/api/auth/login	Public
POST	/api/auth/register	Public
POST	/api/auth/forgot-password	Public
POST	/api/auth/reset-password	Public
POST	/api/auth/change-password	Authenticated

Student

Method	Endpoint	Access
GET	/api/student/dashboard	Student
GET	/api/student/profile	Student
GET/POST	/api/student/study-bot	Student
GET	/api/student/timetable	Student
GET	/api/student/notes	Student
GET	/api/student/reminders	Student
GET	/api/student/parent-report/:studentId	Public

Parent Report

Method	Endpoint	Access
GET	/api/student/parent-report/:studentId	Public QR report

Teacher

Method	Endpoint	Access
POST	/api/teacher/create-test	Teacher
POST	/api/teacher/publish-test	Teacher
GET	/api/teacher/tests	Teacher
POST	/api/teacher/grade-submission	Teacher
GET	/api/teacher/results	Teacher

Admin

Method	Endpoint	Access
POST	/api/admin/create-teacher	Admin
POST	/api/admin/courses	Admin
POST	/api/admin/assign-course	Admin
POST	/api/admin/assign-timetable	Admin
GET	/api/admin/monitor	Admin

Parent QR & Student Report

The Parent QR feature allows a student to share a simple QR code with a parent.

```
Student Dashboard
  ↓
Parent Access QR
  ↓
Scan QR
  ↓
/parent-report/:studentId
  ↓
GET /api/student/parent-report/:studentId
  ↓
Standalone Parent Report
```

Report includes

- Student name
- Email
- Registration number
- Enrolled courses
- Course teacher details
- Test / assessment results
- Marks and percentage
- Grading / submission status

The QR contains the report URL with the student's ID; it does not embed the complete report data inside the QR.

Local development note

A QR generated on localhost contains a localhost URL. Scanning it from a phone normally points to the phone's own localhost. For a phone demo, use the deployed Vercel frontend URL or configure a LAN-accessible development URL.

□ Running Tests

```
cd lms-server
npm test
```

□ Project Structure

```
smart-learn-lms/
├── lms-client/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── admin/
│   │   │   ├── teacher/
│   │   │   ├── student/
│   │   │   ├── parent/
│   │   │   └── auth/
│   │   ├── components/
│   │   ├── context/
│   │   └── routes/
│   └── package.json
├── lms-server/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── utils/
│   └── config/
```

■ ■■■ test/
■ ■■■ package.json
■■■ screenshots/

□ Competition Demo Flow

Recommended order for the final project presentation:

1. Login — demonstrate role-based authentication
2. Student Dashboard — courses, timetable, tests, and Parent QR
3. Parent QR — scan and open the standalone Student Report
4. AI Study Bot — demonstrate AI-assisted learning
5. Teacher Module — courses, resources, tests, grading, messaging
6. Admin Module — teacher/course/timetable management and system overview
7. Responsive UI — show mobile-friendly navigation and layouts

□ Roadmap

- [x] Deploy frontend to Vercel
- [x] Deploy backend to Render
- [x] Implement Parent QR / Student Report
- [x] Complete responsive UI pass
- [] Add CI pipeline for lint/test/build
- [] Improve integration test coverage
- [] Add real-time notifications
- [] Expand parent features with optional authenticated parent accounts

□ Admin Email & Password Change

To change the existing Admin's login email and password, use the `change-admin-password.js` script.

1. Open `lms-server/.env`

Add or update:

```
ADMIN_NEW_EMAIL=newadmin@gmail.com  
ADMIN_NEW_PASSWORD=NewStrongPassword123
```

2. Run the script

```
cd lms-server  
node change-admin-password.js
```

3. Restart the backend

```
npm run dev
```

4. Login with the new credentials

```
Email: newadmin@gmail.com  
Password: NewStrongPassword123
```

Part III — Frontend README

Source: README_lms-client_updated(1).md

□ Smart Learn LMS — Frontend (React)

The frontend application of Smart Learn LMS, a role-based Learning Management System built with React 18, Vite, Tailwind CSS, and Axios.

Image: React

Image: Vite

Image: Tailwind CSS

Image: License

□ Table of Contents

- [Features](#-features)
- [Parent QR & Student Report](#-parent-qr--student-report)
- [Tech Stack](#-tech-stack)
- [Quick Start](#-quick-start)
- [Project Structure](#-project-structure)
- [Components](#-components)
- [State Management](#-state-management)
- [Routing](#-routing)
- [API Integration](#-api-integration)
- [Styling](#-styling)
- [Build & Deploy](#-build--deploy)
- [Contributing](#-contributing)

□ Features

□ User Interface

- Responsive Design: Mobile-first approach with Tailwind CSS
- Modern UI: Clean, professional interface with orange theme
- Role-Based Navigation: Dynamic menus based on user role
- Loading States: Smooth loading indicators and transitions
- Error Handling: User-friendly error messages and boundaries

□□ Student Features

- Dashboard: Today's classes, recent activities, and quick actions
- Course Management: View enrolled courses with resources
- Timetable: Card-based schedule with expandable details
- Assessment: Take tests and view results

- Study Bot: AI assistant for notes and reminders
- Messaging: View teacher announcements
- Profile Management: Update personal information

☐☐ Teacher Features

- Teaching Dashboard: Scheduled classes and student overview
- Course Management: Upload resources and manage content
- Assessment Tools: Create and grade tests
- Messenger: Send messages to students
- Timetable: View teaching schedule
- Student Analytics: Track student progress

☐☐ Admin Features

- User Management: Create and manage teacher accounts
- Course Administration: Assign courses and timetables
- System Monitoring: View analytics and reports
- Profile Management: Manage student and teacher profiles

☐ Authentication

- Multi-Role Login: Single form for all user types
- Password Management: Forgot password, reset, and change
- Session Management: JWT-based authentication
- Route Protection: Role-based access control

☐ Current Project Status

The frontend currently includes the responsive Student, Teacher, and Admin interfaces, role-based routing, AI Study Bot, and the Parent QR / Student Report flow.

Live deployment

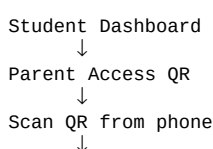
- Frontend: <https://smart-lms-three.vercel.app/>
- Backend API: <https://smart-lms-kbqd.onrender.com/api>

Note: The project is in the final testing / competition-presentation phase. Some backend-side authorization and unfinished admin functionality are documented as future fixes rather than being presented as completed features.

☐ Parent QR & Student Report

The Student Dashboard provides a Parent Access QR code. The QR contains a public frontend URL for the logged-in student's report.

Flow



```
/parent-report/:studentId
↓
Backend fetches student report
↓
Standalone Parent Report page
```

What parents can view

- Student name, email, and registration number
- Enrolled courses
- Course teacher information
- Test / assessment results
- Marks, percentage, submission date, and grading status

Frontend route

```
/parent-report/:studentId
```

This route is intentionally placed outside the authenticated dashboard layout, so parents see a standalone report instead of the Student / Teacher / Admin sidebar.

QR generation

The Student Dashboard uses `QRCodeCanvas` from `qrcode.react`:

```
const parentReportUrl = user?.id
  ? `${window.location.origin}/parent-report/${user.id}`
  : '';
```

The QR therefore automatically uses the current frontend origin. On the deployed site, it points to the Vercel application.

Important local-development note

If the QR is generated while running the frontend on `localhost`, the QR contains a `localhost` URL. Scanning that QR from a phone will point to the phone's own `localhost` and normally will not open the developer's PC.

For a real phone demo, use the deployed Vercel URL or configure a LAN-accessible development URL.

📦 Tech Stack

Core Technologies

- React 18.2.0 - Modern UI library with hooks
- React Router DOM 6.22.3 - Client-side routing
- Vite 4.5.0 - Fast build tool and dev server
- Tailwind CSS 3.4.3 - Utility-first CSS framework

Additional Libraries

- Axios 1.4.0 - HTTP client for API communication
- React Icons 5.5.0 - Comprehensive icon library
- React Context - State management for authentication

- qrcode.react - Generates the student Parent Access QR code

Development Tools

- ESLint - Code linting
- Prettier - Code formatting
- Error Boundaries - React error handling

⚡ Quick Start

Prerequisites

- Node.js (v16 or higher)
- npm or yarn package manager

Installation

```
git clone https://github.com/bs-bhaskar/smart-lms.git
cd smart-lms/lms-client
npm install
cp .env.example .env
nano .env
```

Environment Variables

```
VITE_API_BASE_URL=http://localhost:5000/api
```

Development Server

```
npm run dev
```

Build for Production

```
npm run build
npm run preview
```

📁 Project Structure

```
src/
├── components/                # Reusable components
│   ├── shared/                # Shared components
│   │   ├── Navbar.jsx        # Navigation bar
│   │   ├── Sidebar.jsx       # Sidebar navigation
│   │   ├── Loader.jsx        # Loading spinner
│   │   └── ErrorBoundary.jsx  # Error handling
│   └── pages/                # Page components
│       ├── auth/              # Authentication pages
│       │   ├── Login.jsx      # Login form
│       │   ├── Register.jsx   # Student registration
│       │   ├── ForgotPassword.jsx # Password reset
│       │   ├── ResetPassword.jsx # Password reset form
│       │   └── ChangePassword.jsx # Change password
│       ├── student/           # Student pages
│       │   ├── Dashboard.jsx  # Student dashboard
│       │   ├── Courses.jsx     # Course management
│       │   ├── Timetable.jsx   # Personal timetable
│       │   ├── Profile.jsx     # Student profile
│       │   ├── StudyBot.jsx    # AI study assistant
│       │   ├── Messages.jsx    # Teacher messages
│       │   ├── Tests.jsx       # Available tests
│       │   └── Results.jsx     # Test results
│       ├── parent/            # Public parent report
│       │   ├── ParentReport.jsx # Standalone student report
│       └── teacher/           # Teacher pages
│           ├── Dashboard.jsx   # Teacher dashboard
│           ├── Courses.jsx     # Course management
│           ├── AddMaterial.jsx  # Resource upload
│           └── Messenger.jsx    # Student messaging
```


■ ■ ■ ■	Profile.jsx	# Teacher profile
■ ■ ■ ■	Tests.jsx	# Test management
■ ■ ■ ■	Results.jsx	# Grade management
■ ■ ■ ■	admin/	# Admin pages
■ ■ ■ ■	Dashboard.jsx	# Admin dashboard
■ ■ ■ ■	CreateTeacher.jsx	# Teacher creation
■ ■ ■ ■	ManageProfiles.jsx	# User management
■ ■ ■ ■	UploadResources.jsx	# Resource management
■ ■ ■ ■	AssignCourses.jsx	# Course assignment
■ ■ ■ ■	AssignTimetables.jsx	# Timetable assignment
■ ■ ■ ■	Results.jsx	# System analytics
■ ■ ■ ■	routes/	# Route protection
■ ■ ■ ■	PrivateRoute.jsx	# Authentication guard
■ ■ ■ ■	TeacherRoute.jsx	# Teacher access control
■ ■ ■ ■	AdminRoute.jsx	# Admin access control
■ ■ ■ ■	context/	# React context
■ ■ ■ ■	AuthContext.jsx	# Authentication state
■ ■ ■ ■	api/	# API configuration
■ ■ ■ ■	axios.js	# Axios instance
■ ■ ■ ■	App.jsx	# Main app component
■ ■ ■ ■	main.jsx	# App entry point
■ ■ ■ ■	index.css	# Global styles

□ Components

Shared Components

Navbar

- Purpose: Main navigation bar with role-based menu
- Features: Logo, navigation links, logout button
- Props: None (uses AuthContext)

Sidebar

- Purpose: Side navigation for authenticated users
- Features: Role-based menu items, active state
- Props: None (uses AuthContext and useLocation)

Loader

- Purpose: Loading spinner component
- Features: Centered spinner with optional text
- Props: text (optional)

ErrorBoundary

- Purpose: React error boundary for error handling
- Features: Catches JavaScript errors, displays fallback UI
- Props: children

Page Components

Authentication Pages

- Login: Multi-role login with email/registration number
- Register: Student self-registration
- ForgotPassword: Email-based password reset
- ResetPassword: OTP verification and password reset

- ChangePassword: In-app password modification

Student Pages

- Dashboard: Overview with today's classes and activities
- Courses: Enrolled courses with resources and comments
- Timetable: Personal schedule with card-based UI
- Profile: Personal information and settings
- StudyBot: AI assistant for notes and reminders
- Messages: Teacher announcements and notifications
- Tests: Available tests with expiry information
- Results: Test results and performance history

Parent Report Page

- ParentReport: Public, standalone student academic report opened from the Parent QR code
- Data source: Fetches report data from the backend using the student ID in the URL

Teacher Pages

- Dashboard: Teaching overview with scheduled classes
- Courses: Course management with resource upload
- AddMaterial: File and link upload interface
- Messenger: Student communication system
- Profile: Teacher information and settings
- Tests: Test creation and management
- Results: Student grading and analytics

Admin Pages

- Dashboard: System overview and management
- CreateTeacher: Teacher account creation
- ManageProfiles: User management interface
- UploadResources: System-wide resource management
- AssignCourses: Course assignment interface
- AssignTimetables: Timetable management
- Results: System analytics and reports

□ State Management

Authentication Context

```
{
  user: User object,
  role: 'student' | 'teacher' | 'admin',
  token: JWT token,
  loading: boolean,
  login: function,
  logout: function,
  updateUser: function
}
```

Local State

- useState: Component-level state management
- useEffect: Side effects and API calls
- Custom hooks: Reusable state logic

State Patterns

- Loading states: Consistent loading indicators
- Error handling: User-friendly error messages

□ Routing

Route Structure

```
// Public routes
/login
/register
/forgot-password
/reset-password

// Public parent report
/parent-report/:studentId

// Protected routes
/student/dashboard
/student/courses
/student/timetable
/student/profile
/student/studybot
/student/messages
/student/tests
/student/results

/teacher/dashboard
/teacher/courses
/teacher/add-material
/teacher/messenger
/teacher/profile
/teacher/tests
/teacher/results

/admin/dashboard
/admin/create-teacher
/admin/manage-profiles
/admin/upload-resources
/admin/assign-courses
/admin/assign-timetables
/admin/results
```

Route Protection

- PrivateRoute: Requires authentication
- TeacherRoute: Requires teacher role
- AdminRoute: Requires admin role
- Automatic redirects: Based on user role

□ API Integration

Axios Configuration

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 10000,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('lmsToken');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('lmsToken');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

API Patterns

- Consistent error handling: Centralized error management
- Loading states: Automatic loading indicators
- API error handling: Centralized Axios response handling
- Authenticated requests: JWT attached through Axios interceptor

□ Styling

Tailwind CSS

- Utility-first: Rapid UI development
- Responsive design: Mobile-first approach
- Custom theme: Orange color scheme
- Component classes: Reusable style patterns

Design System

```
:root {
  --color-orange: #ff6b35;
  --color-orange-dark: #e55a2b;
}

.btn-primary {
  @apply bg-orange text-white px-4 py-2 rounded hover:bg-orange-dark transition;
}

.card {
  @apply bg-white rounded-xl shadow p-6;
}
```

Responsive Design

- Mobile-first: Base styles for mobile
- Tablet: Medium breakpoint styles
- Desktop: Large breakpoint styles
- Accessibility: WCAG compliant design

□ Build & Deploy

Development

```
npm run dev
npm run lint
npm run format # if configured in package.json
```

Production Build

```
npm run build
npm run preview
npm run analyze # if configured in package.json
```

Deployment Options

Vercel (Recommended)

```
npm i -g vercel
vercel
```

Netlify

```
npm run build
# Upload dist/ folder to Netlify
```

Static Hosting

```
npm run build
npx serve dist
```

Environment Variables

```
# Development
VITE_API_BASE_URL=http://localhost:5000/api

# Production
VITE_API_BASE_URL=https://smart-lms-kbqd.onrender.com/api
```

□ Contributing

Development Setup

1. Fork the repository
2. Create a feature branch
3. Install dependencies: `npm install`
4. Start development server: `npm run dev`
5. Make your changes
6. Run tests: `npm test` # if configured in package.json
7. Submit a pull request

Code Style

- Use ESLint and Prettier
- Follow React best practices
- Write meaningful commit messages
- Add comments for complex logic

Testing

```
npm test # if configured in package.json
npm run test:coverage # if configured in package.json
npm run test:watch # if configured in package.json
```

□ Parent QR Testing

1. Log in as a student.
2. Open Student Dashboard.
3. Confirm the Parent Access QR is visible.
4. Scan the QR using a phone.
5. Confirm `/parent-report/:studentId` opens without login.
6. Verify student details, courses, and results shown by the backend.
7. Test an invalid student ID and confirm a suitable error state is displayed.

□ Performance

Optimization Techniques

- Code splitting: Route-based code splitting
- Lazy loading: Component lazy loading
- Image optimization: Optimized image loading
- Bundle analysis: Regular bundle size monitoring

Performance Notes

Performance depends on the deployed frontend, backend response time, device, and network conditions. Use Lighthouse and the browser DevTools for final production measurements.

□ Security

Security Measures

- Input validation: Client-side validation
- XSS prevention: React's built-in protection
- HTTPS in production: Frontend and backend are deployed over HTTPS

Best Practices

- Environment variables: Secure configuration
- Error handling: No sensitive data in errors
- Authentication: Secure token management
- File uploads: Type and size validation

□ Analytics

User Analytics

- Page views: Track user navigation

- Feature usage: Monitor feature adoption
- Error tracking: Monitor application errors
- Performance monitoring: Track load times

Implementation

Application-level error handling is provided through React error boundaries and Axios response handling. External analytics can be added later if required.

☐ Troubleshooting

Common Issues

Build Errors

```
rm -rf node_modules package-lock.json
npm install
```

API Connection Issues

```
echo $VITE_API_BASE_URL
curl http://localhost:5000/api/health
```

Styling Issues

```
npm run build # rebuild Tailwind/Vite output
# Hard refresh: Ctrl+Shift+R
```

☐ Resources

Documentation

- [React Documentation](https://react.dev/)
- [Vite Documentation](https://vitejs.dev/)
- [Tailwind CSS Documentation](https://tailwindcss.com/)
- [React Router Documentation](https://reactrouter.com/)

Tools

- [React Developer Tools](https://chrome.google.com/webstore/detail/react-developer-tools)
- [Tailwind CSS IntelliSense](https://marketplace.visualstudio.com/items?itemName=bradlc.vscode-tailwindcss)

☐ Competition Demo Flow

A concise frontend demo can follow this order:

1. Student login and dashboard
2. Responsive navigation on desktop/mobile
3. Courses, timetable, tests, and results
4. AI Study Bot
5. Parent Access QR → scan → standalone Parent Report

- 6. Teacher dashboard and assessment workflow
- 7. Admin dashboard and management features

📄 License

This project is licensed under the MIT License - see the [LICENSE](../LICENSE) file for details.

Built with ❤️ using React and Tailwind CSS

[![GitHub stars](https://img.shields.io/github/stars/yourusername/lms-project?style=social)](https://github.com/bs-bhaskar/smart-lms/stargazers)

[![GitHub issues](https://img.shields.io/github/issues/yourusername/lms-project)](https://github.com/bs-bhaskar/smart-lms/issues)

Part IV — Backend README

Source: README_updated(1).md

□ LMS Backend (Node.js)

The backend API server for the Learning Management System built with Node.js, Express, and MongoDB.

Image: Node.js

Image: Express

Image: MongoDB

Image: JWT

Image: License

□ Current Project Status

The backend supports the core LMS modules for authentication, courses, assessments, timetable management, communication, resources, AI integration, and the Parent QR / Student Report workflow.

The Parent Report endpoint is designed to work without a parent account because access is initiated from the student's QR-generated report URL.

□ Table of Contents

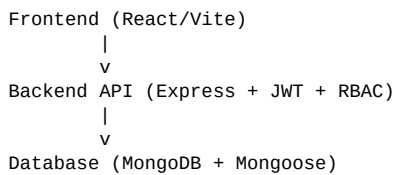
- [Features](#-features)
- [Parent QR & Student Report](#-parent-qr--student-report)
- [Tech Stack](#-tech-stack)
- [Quick Start](#-quick-start)
- [Project Structure](#-project-structure)
- [API Documentation](#-api-documentation)
- [Database Schema](#-database-schema)
- [Authentication](#-authentication)
- [File Upload](#-file-upload)
- [Email System](#-email-system)
- [Security](#-security)
- [Testing](#-testing)
- [Deployment](#-deployment)
- [Contributing](#-contributing)

□ Features

□ Project Overview

Smart Learn LMS is a role-aware backend API for students, teachers, and administrators. It handles secure authentication, course/timetable management, assessments, and messaging while enforcing strict role-based access at the route level.

□ Architecture Diagram



□ Authentication Flow

1. User submits credentials to a role-specific login route.
2. Server validates credentials and signs a JWT with user id + role.
3. Client sends JWT in Authorization: Bearer <token>.
4. authenticateJWT verifies signature and injects req.user.
5. authorizeRoles(...) enforces route-level permissions.

□ Security Features

- JWT Authentication
- Role-Based Access Control (RBAC)
- Environment Variable Protection (startup validation + no JWT fallback secret)
- First Admin Bootstrap Protection (/api/admin/create-first-admin only works if no admin exists)
- Rate limiting for login endpoints
- Input validation for auth/password flows
- Centralized error handling and structured request/error logging
- Secure HTTP headers middleware (helmet-equivalent hardening)
- Mongo query/body sanitization against operator injection
- CORS restriction with environment-based client allowlist

□ Security Architecture

Request

- > CORS policy check
- > Secure headers middleware
- > Mongo input sanitization
- > JWT authentication (where required)
- > Role authorization (RBAC)
- > Route controller
- > Centralized error handler

□ Threat Mitigation

- Prevented privilege escalation by binding sensitive operations to JWT identity (req.user.id).
- Eliminated fallback JWT secret usage by failing fast when JWT_SECRET is missing.
- Enforced role-based access for admin router via middleware at router level.

- Added login rate limiting to reduce brute-force attack surface.
- Sanitized request body/query/params to block NoSQL operator injection payloads.

□ **Sample Protected Route**

```
router.use(authenticateJWT, authorizeRoles('admin'));
router.get('/teachers', adminController.listTeachers);
```

All routes after `router.use( . . . )` are protected and require a valid admin token.

□ **Authentication & Authorization**

- JWT-based Authentication: Secure token-based sessions
- Role-Based Access Control: Student, Teacher, Admin permissions
- Password Security: bcrypt hashing with salt rounds
- Password Reset: Email-based OTP verification
- Session Management: Automatic token expiration

□ **Course Management**

- Course CRUD: Create, read, update, delete courses
- Teacher Assignment: Assign teachers to courses
- Student Enrollment: Enroll students in courses
- Resource Management: File and link uploads
- Course Comments: Interactive feedback system

□ **Timetable System**

- Schedule Creation: Flexible timetable setup
- Time Slot Management: Multiple time slots per day
- Conflict Detection: Automated schedule validation
- Bulk Assignment: Mass timetable operations
- Day-wise Organization: Weekday-based structure

□ **Assessment System**

- Test Creation: MCQ and theory-based tests
- Question Management: Multiple question types
- Auto-Grading: Instant MCQ result calculation
- Manual Grading: Teacher grading for theory questions
- Result Analytics: Performance tracking and reports
- Expiry Management: Time-limited test access

□ **Communication**

- Messenger System: Teacher-student communication
- Message Broadcasting: Send messages to multiple recipients
- Notification System: Real-time updates
- Email Integration: Automated email notifications

□ **User Management**

- Multi-Role Users: Student, Teacher, Admin accounts
- Profile Management: Complete user profile CRUD
- Registration System: Student self-registration
- Admin Creation: Secure teacher account creation
- Welcome Emails: Automated account notifications

□ **Parent QR & Student Report**

- Parent Report API: Public endpoint for QR-based student report access
- Student Information: Name, email, and registration number
- Course Information: Enrolled courses with teacher details
- Assessment Results: Test marks, percentage, and grading status
- QR-Based Access: Student dashboard generates a report URL that can be encoded as a QR code
- No Parent Login Required: Report is accessed directly through the QR-generated URL

□ **File Management**

- File Upload: Secure file storage system
- Type Validation: Restricted file types
- Size Limits: Configurable file size restrictions
- Access Control: Role-based file access
- Static Serving: Efficient file delivery

□ **Tech Stack**

Core Technologies

- Node.js - JavaScript runtime environment
- Express.js 5.1.0 - Web application framework
- MongoDB 8.16.4 - NoSQL database
- Mongoose - MongoDB object modeling

Authentication & Security

- JWT 9.0.2 - JSON Web Tokens
- bcryptjs 3.0.2 - Password hashing
- CORS 2.8.5 - Cross-origin resource sharing

File Handling

- Multer 2.0.2 - File upload middleware
- Path - File path utilities
- FS - File system operations

Communication

- Nodemailer 7.0.5 - Email functionality
- SMTP - Email transport

Development Tools

- Nodemon - Development server with auto-restart
- ESLint - Code linting
- Prettier - Code formatting

⚡ Quick Start

Prerequisites

- Node.js (v16 or higher)
- MongoDB (v4.4 or higher)
- npm or yarn package manager

Installation

```
git clone https://github.com/bs-bhaskar/smart-lms.git
cd lms-project/lms-server
npm install
cp .env.example .env
nano .env
```

Environment Variables

```
PORT=5000
NODE_ENV=development
MONGO_URI=mongodb://localhost:27017/lms
JWT_SECRET=your-super-secret-jwt-key-here
JWT_EXPIRES_IN=7d
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your-email@gmail.com
EMAIL_PASS=your-app-password
MAX_FILE_SIZE=10485760
UPLOAD_PATH=./uploads
BCRYPT_ROUNDS=10
```

Database Setup

```
mongod
npm run setup-db
npm run setup-admin
```

Development Server

```
npm run dev
```

Production Server

```
npm start
npm install -g pm2
pm2 start index.js --name "lms-backend"
```

📁 Project Structure

```
lms-server/
├── controllers/           # Business logic
│   ├── adminController.js # Admin operations
│   ├── studentController.js # Student + parent report operations
│   └── teacherController.js # Teacher operations
```

■ ■■■ testController.js	# Test management
■■■ models/	# Database schemas
■ ■■■ User.js	# User model
■ ■■■ Course.js	# Course model
■ ■■■ Timetable.js	# Timetable model
■ ■■■ Test.js	# Test model
■ ■■■ Submission.js	# Test submission model
■ ■■■ Message.js	# Message model
■ ■■■ Note.js	# Note model
■ ■■■ Reminder.js	# Reminder model
■■■ routes/	# API endpoints
■ ■■■ auth.js	# Authentication routes
■ ■■■ student.js	# Student routes
■ ■■■ teacher.js	# Teacher routes
■ ■■■ admin.js	# Admin routes
■■■ middleware/	# Custom middleware
■ ■■■ auth.js	# JWT authentication
■ ■■■ requestLogger.js	# Structured request logging
■ ■■■ rateLimit.js	# Login rate limiting
■ ■■■ mongoSanitize.js	# NoSQL injection sanitization
■ ■■■ secureHeaders.js	# Security headers hardening
■ ■■■ errorHandler.js	# Centralized error handling
■■■ utils/	# Helper functions
■ ■■■ email.js	# Email utilities
■ ■■■ password.js	# Password utilities
■ ■■■ validation.js	# Validation helpers
■ ■■■ response.js	# Response formatting
■■■ uploads/	# File storage
■ ■■■ resources/	# Course resources
■ ■■■ profiles/	# Profile pictures
■ ■■■ temp/	# Temporary files
■■■ config/	# Configuration files
■ ■■■ database.js	# Database configuration
■ ■■■ email.js	# Email configuration
■ ■■■ multer.js	# File upload configuration
■■■ index.js	# Server entry point
■■■ setup-admin.js	# Admin user creation
■■■ package.json	# Dependencies and scripts
■■■ .env	# Environment variables

□ API Documentation

Authentication Endpoints

Login

POST /api/auth/login
Content-Type: application/json

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Register (Student)

POST /api/auth/register
Content-Type: application/json

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "password123"
}
```

Forgot Password

POST /api/student/forgot-password
POST /api/teacher/forgot-password
Content-Type: application/json

```
{
  "email": "user@example.com"
}
```

Reset Password

POST /api/student/reset-password
POST /api/teacher/reset-password
Content-Type: application/json

```
{
  "email": "user@example.com",
  "otp": "123456",
  "newPassword": "newpassword123"
}
```

Student Endpoints

Dashboard

GET /api/student/dashboard
Authorization: Bearer <token>

Profile

GET /api/student/profile
Authorization: Bearer <token>

Courses

GET /api/student/courses
Authorization: Bearer <token>

Timetable

GET /api/student/timetable
Authorization: Bearer <token>

Tests

GET /api/teacher/student-tests
Authorization: Bearer <token>

Submit Test

POST /api/teacher/submit-test
Authorization: Bearer <token>
Content-Type: application/json

```
{
  "testId": "test_id",
  "answers": [
    {
      "questionId": "question_id",
      "answer": "student_answer"
    }
  ]
}
```

Parent Report Endpoint

Get Student Report

GET /api/student/parent-report/:studentId

This endpoint is intentionally placed before the authenticated student routes so that the report can be opened directly after scanning the Parent QR.

Example response:

```
{
  "student": {
    "name": "Student Name",
    "email": "student@example.com",
  }
}
```

```

    "registrationNumber": "123456"
  },
  "courses": [],
  "results": []
}

```

Parent QR Flow

```

Student Dashboard
  ↓
Parent QR contains report URL
  ↓
Parent scans QR
  ↓
/parent-report/:studentId
  ↓
Frontend requests:
GET /api/student/parent-report/:studentId
  ↓
Backend fetches student + courses + submissions
  ↓
Standalone Student Report

```

Teacher Endpoints

Dashboard

```

GET /api/teacher/dashboard
Authorization: Bearer <token>

```

Courses

```

GET /api/teacher/courses
Authorization: Bearer <token>

```

Upload Resource

```

POST /api/teacher/upload-resource
Authorization: Bearer <token>
Content-Type: multipart/form-data

```

```

{
  "courseId": "course_id",
  "file": <file>
}

```

Add Comment

```

POST /api/teacher/add-comment
Authorization: Bearer <token>
Content-Type: application/json

```

```

{
  "courseId": "course_id",
  "text": "Comment text"
}

```

Messenger

```

GET /api/teacher/messenger
Authorization: Bearer <token>

```

Send Message

```

POST /api/teacher/messenger
Authorization: Bearer <token>
Content-Type: application/json

```

```

{
  "content": "Message content",
  "recipients": ["student_id1", "student_id2"]
}

```


Admin Endpoints

Create Teacher

POST /api/admin/create-teacher
Authorization: Bearer <token>
Content-Type: application/json

```
{
  "name": "Teacher Name",
  "email": "teacher@example.com",
  "password": "password123"
}
```

List Users

GET /api/admin/teachers
GET /api/admin/students
Authorization: Bearer <token>

Assign Course

POST /api/admin/assign-course
Authorization: Bearer <token>
Content-Type: application/json

```
{
  "courseId": "course_id",
  "teacherId": "teacher_id",
  "studentIds": ["student_id1", "student_id2"]
}
```

Assign Timetable

POST /api/admin/assign-timetable
Authorization: Bearer <token>
Content-Type: application/json

```
{
  "courseId": "course_id",
  "schedule": [
    {
      "day": "Monday",
      "startTime": "09:00",
      "endTime": "10:00",
      "notes": "Optional notes"
    }
  ]
}
```

□ Database Schema

User Model

```
{
  name: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    index: true
  },
  password: {
    type: String,
    required: true,
    minlength: 6
  },
  registrationNumber: {
    type: String,
    required: true,
    unique: true,
    index: true
  },
  role: {
    type: String,
    enum: ['student', 'teacher', 'admin'],
    required: true
  },
  emailVerified: {
    type: Boolean,
    default: false
  },
  otp: String,
  otpExpires: Date,
  createdAt: {
    type: Date,
    default: Date.now
  },
  updatedAt: {
    type: Date,
    default: Date.now
  }
}
```

Course Model

```
{
  title: {
    type: String,
    required: true,
    trim: true
  },
  description: {
    type: String,
    trim: true
  },
  teacher: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  students: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  }],
  resources: [{
    filename: String,
    originalName: String,
    url: String,
    uploadedAt: {
      type: Date,
      default: Date.now
    }
  }],
  timetable: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Timetable'
  },
  comments: [{
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    },
    text: String,
    createdAt: {
      type: Date,
      default: Date.now
    }
  }],
  createdAt: {
    type: Date,
    default: Date.now
  },
  updatedAt: {
    type: Date,
    default: Date.now
  }
}
```

Test Model

```
{
  title: {
    type: String,
    required: true,
    trim: true
  },
  description: {
    type: String,
    trim: true
  },
  course: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Course',
    required: true
  },
  questions: [{
    question: {
      type: String,
      required: true
    },
    type: {
      type: String,
      enum: ['mcq', 'theory'],
      required: true
    },
    options: [String],
    correctAnswer: String,
    marks: {
      type: Number,
      required: true,
      min: 1
    }
  }
  ],
  duration: {
    type: Number,
    required: true,
    min: 1
  },
  published: {
    type: Boolean,
    default: false
  },
  expiryDate: Date,
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  },
  updatedAt: {
    type: Date,
    default: Date.now
  }
}
```

□ Authentication

JWT Implementation

```
const generateToken = (userId) => {
  return jwt.sign(
    { id: userId },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN }
  );
};

const verifyToken = (token) => {
  try {
    return jwt.verify(token, process.env.JWT_SECRET);
  } catch (error) {
    throw new Error('Invalid token');
  }
};
```

Middleware

```
const authenticateJWT = async (req, res, next) => {
  try {
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) {
      return res.status(401).json({ message: 'Access denied' });
    }

    const decoded = verifyToken(token);
    const user = await User.findById(decoded.id);
    if (!user) {
      return res.status(401).json({ message: 'User not found' });
    }

    req.user = user;
    next();
  } catch (error) {
    res.status(401).json({ message: 'Invalid token' });
  }
};

const authorizeRoles = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        message: 'Access denied for this role'
      });
    }
    next();
  };
};
```

Password Security

```
const hashPassword = async (password) => {
  const saltRounds = parseInt(process.env.BCRYPT_ROUNDS) || 10;
  return await bcrypt.hash(password, saltRounds);
};

const verifyPassword = async (password, hashedPassword) => {
  return await bcrypt.compare(password, hashedPassword);
};
```

□ File Upload

Multer Configuration

```
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, process.env.UPLOAD_PATH || './uploads');
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, file.fieldname + '-' + uniqueSuffix + path.extname(file.originalname));
  }
});

const fileFilter = (req, file, cb) => {
  const allowedTypes = [
    'image/jpeg',
    'image/png',
    'image/gif',
    'application/pdf',
    'application/msword',
    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
    'video/mp4',
    'video/webm'
  ];

  if (allowedTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('Invalid file type'), false);
  }
};

const upload = multer({
  storage: storage,
  limits: {
    fileSize: parseInt(process.env.MAX_FILE_SIZE) || 10 * 1024 * 1024
  },
  fileFilter: fileFilter
});
```

Upload Endpoints

```
router.post('/upload-resource',
  authenticateJWT,
  authorizeRoles('teacher', 'admin'),
  upload.single('file'),
  uploadResource
);

router.post('/upload-multiple',
  authenticateJWT,
  authorizeRoles('teacher', 'admin'),
  upload.array('files', 5),
  uploadMultipleResources
);
```

□ Email System

Email Configuration

```
const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransporter({
  host: process.env.EMAIL_HOST,
  port: process.env.EMAIL_PORT,
  secure: false,
  auth: {
    user: process.env.EMAIL_USER,
    pass: process.env.EMAIL_PASS
  }
});
```

Email Templates

```
const welcomeEmailTemplate = (name, email, password, registrationNumber) => `  
  Dear ${name},  
  
  Welcome to the Learning Management System!  
  
  Your account has been successfully created with the following details:  
  
  Name: ${name}  
  Email: ${email}  
  Registration Number: ${registrationNumber}  
  Password: ${password}  
  
  You can now log in to your account using your email or registration number.  
  For security, please change your password after your first login.  
  
  Best regards,  
  LMS Administration Team  
`;  
  
const passwordResetTemplate = (otp) => `  
  Your password reset OTP is: ${otp}  
  
  This OTP will expire in 10 minutes.  
  If you didn't request this, please ignore this email.  
`;
```

Email Functions

```
const sendEmail = async (to, subject, html) => {  
  try {  
    const mailOptions = {  
      from: process.env.EMAIL_USER,  
      to: to,  
      subject: subject,  
      html: html  
    };  
  
    await transporter.sendMail(mailOptions);  
    console.log('Email sent successfully');  
  } catch (error) {  
    console.error('Email sending failed:', error);  
    throw error;  
  }  
};  
  
const sendWelcomeEmail = async (user, password) => {  
  const subject = 'Welcome to LMS - Account Created';  
  const html = welcomeEmailTemplate(  
    user.name,  
    user.email,  
    password,  
    user.registrationNumber  
  );  
  
  await sendEmail(user.email, subject, html);  
};
```

□ Security

Security Measures

- Input Validation: Comprehensive data validation
- SQL Injection Prevention: Mongoose ORM protection
- XSS Protection: Input sanitization
- CSRF Protection: Token-based validation
- Rate Limiting: Request throttling
- CORS Configuration: Cross-origin protection

Security Middleware

```
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

app.use(helmet());

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100
});
app.use('/api/', limiter);

app.use(cors({
  origin: process.env.FRONTEND_URL || 'http://localhost:5173',
  credentials: true
}));
```

Error Handling

```
app.use((error, req, res, next) => {
  console.error(error.stack);

  if (error.name === 'ValidationError') {
    return res.status(400).json({
      message: 'Validation Error',
      errors: Object.values(error.errors).map(err => err.message)
    });
  }

  if (error.name === 'CastError') {
    return res.status(400).json({
      message: 'Invalid ID format'
    });
  }

  res.status(500).json({
    message: 'Internal Server Error'
  });
});
```

□ Testing

Test Setup

```
npm install --save-dev jest supertest
npm test
npm run test:coverage
npm run test:watch
```

Parent QR Testing

Verify the complete flow:

1. Log in as a student.
2. Open the Student Dashboard.
3. Confirm the Parent QR is visible.
4. Scan the QR using a phone.
5. Confirm the standalone Parent Report opens.
6. Verify student details.
7. Verify enrolled courses when available.
8. Verify test results when submissions exist.

For real phone testing, use the deployed frontend URL. A QR generated from a local localhost URL will normally not open the computer's local server from another phone.

Test Examples

```
describe('POST /api/auth/login', () => {
  it('should login with valid credentials', async () => {
    const response = await request(app)
      .post('/api/auth/login')
      .send({
        email: 'test@example.com',
        password: 'password123'
      });

    expect(response.status).toBe(200);
    expect(response.body).toHaveProperty('token');
    expect(response.body).toHaveProperty('user');
  });
});

describe('GET /api/student/courses', () => {
  it('should return enrolled courses', async () => {
    const response = await request(app)
      .get('/api/student/courses')
      .set('Authorization', `Bearer ${token}`);

    expect(response.status).toBe(200);
    expect(Array.isArray(response.body.courses)).toBe(true);
  });
});
```

Deployment

Production Setup

```
npm install -g pm2
pm2 start index.js --name "lms-backend"
pm2 save
pm2 startup
pm2 monit
```

Environment Variables (Production)

```
NODE_ENV=production
PORT=5000
MONGO_URI=mongodb://your-mongodb-uri
JWT_SECRET=your-production-jwt-secret
JWT_EXPIRES_IN=7d
EMAIL_HOST=smtplib.org
EMAIL_PORT=587
EMAIL_USER=your-email@gmail.com
EMAIL_PASS=your-app-password
MAX_FILE_SIZE=10485760
UPLOAD_PATH=/var/www/uploads
```

Docker Deployment

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production

COPY . .

EXPOSE 5000

CMD ["npm", "start"]

version: '3.8'
services:
  backend:
    build: .
    ports:
      - "5000:5000"
    environment:
      - NODE_ENV=production
```

```

    - MONGO_URI=mongodb://mongo:27017/lms
  depends_on:
    - mongo
  volumes:
    - ./uploads:/app/uploads

mongo:
  image: mongo:8.16.4
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db

volumes:
  mongo_data:

```

Current Project Deployment

Frontend: Vercel
 Backend: Render
 Database: MongoDB Atlas

The frontend uses VITE_API_BASE_URL to communicate with the deployed backend API.

Cloud Deployment

Heroku

```

heroku create your-lms-backend
heroku config:set NODE_ENV=production
heroku config:set MONGO_URI=your-mongodb-uri
heroku config:set JWT_SECRET=your-jwt-secret
git push heroku main

```

AWS

```

ssh -i your-key.pem ubuntu@your-ec2-instance
git clone https://github.com/bs-bhaskar/smart-lms.git
cd lms-server
npm install
pm2 start index.js --name "lms-backend"

```

📊 Monitoring

Health Check

```

app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'OK',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    memory: process.memoryUsage(),
    environment: process.env.NODE_ENV
  });
});

```

Logging

```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});

if (process.env.NODE_ENV !== 'production') {
  logger.add(new winston.transports.Console({
    format: winston.format.simple()
  }));
}
```

□ Contributing

Development Setup

1. Fork the repository
2. Create a feature branch
3. Install dependencies: `npm install`
4. Set up environment variables
5. Start development server: `npm run dev`
6. Make your changes
7. Run tests: `npm test`
8. Submit a pull request

Code Style

- Use ESLint and Prettier
- Follow Node.js best practices
- Write meaningful commit messages
- Add comments for complex logic
- Include error handling

API Documentation

- Update API documentation for new endpoints
- Include request/response examples
- Document error codes and messages
- Add authentication requirements

□ Performance

Optimization Techniques

- Database Indexing: Optimized MongoDB queries
- Connection Pooling: Efficient database connections

- Caching: Redis for frequently accessed data
- Compression: Gzip compression for responses
- File Streaming: Efficient file uploads/downloads

Performance Monitoring

```
const responseTime = require('response-time');

app.use(responseTime((req, res, time) => {
  console.log(`${req.method} ${req.url} - ${time}ms`);
})));
```

□ Maintenance

Database Maintenance

```
const cleanupExpiredOTPs = async () => {
  await User.updateMany(
    { otpExpires: { $lt: new Date() } },
    { $unset: { otp: 1, otpExpires: 1 } }
  );
};

setInterval(cleanupExpiredOTPs, 24 * 60 * 60 * 1000);
```

File Cleanup

```
const fs = require('fs');
const path = require('path');

const cleanupTempFiles = async () => {
  const tempDir = path.join(__dirname, 'uploads', 'temp');
  const files = fs.readdirSync(tempDir);

  files.forEach(file => {
    const filePath = path.join(tempDir, file);
    const stats = fs.statSync(filePath);

    if (Date.now() - stats.mtime.getTime() > 24 * 60 * 60 * 1000) {
      fs.unlinkSync(filePath);
    }
  });
};
```

□ Resources

Documentation

- [Node.js Documentation](https://nodejs.org/docs/)
- [Express.js Documentation](https://expressjs.com/)
- [MongoDB Documentation](https://docs.mongodb.com/)
- [Mongoose Documentation](https://mongoosejs.com/docs/)

Tools

- [Postman](https://www.postman.com/) - API testing
- [MongoDB Compass](https://www.mongodb.com/products/compass) - Database GUI
- [PM2](https://pm2.keymetrics.io/) - Process manager

□ License

This project is licensed under the MIT License - see the [LICENSE](../LICENSE) file for details.

Built with ♥ using Node.js and Express

[![GitHub stars](https://img.shields.io/github/stars/bs-bhaskar/lms-project?style=social)](https://github.com/bs-bhaskar/smart-lms/stargazers)

[![GitHub issues](https://img.shields.io/github/issues/yourusername/lms-project)](https://github.com/bs-bhaskar/smart-lms/issues)